Universidad Nacional Autónoma de México
Facultad de Ingeniería

Tarea 8

Bases de Datos (1644)

Profesor: Ing. Fernando Arreola Franco

Semestre 2026-2

Grupo: 1

Alumna: Cruz Basilio Ximena Carolina

No. de cuenta: 321116424

Fecha de entrega: 13 de abril de 2026

I. Tipos de datos numéricos en PostgreSQL

Los tipos de datos numéricos en PostgreSQL permiten almacenar valores enteros, valores con decimales exactos y valores aproximados. La elección del tipo adecuado depende principalmente de la precisión requerida, el rango de valores y el espacio de almacenamiento disponible [1], [2].

I-A. Clasificación general

PostgreSQL divide los tipos numéricos en tres grupos principales:

- Enteros (sin decimales)
- Exactos (con decimales precisos)
- Aproximados (punto flotante)

I-B. 1. Tipos enteros

Se utilizan para almacenar números sin parte decimal.

- **smallint (int2)**
  - **Capacidad:** 2 bytes
  - **Rango:** -32,768 a 32,767
  - **Formato:** números enteros (ej. 25, -10)
  - **Ejemplo:**
    ```
    edad smallint
    ```
  - **Uso:** valores pequeños donde se quiere ahorrar espacio.
- **integer (int, int4)**
  - **Capacidad:** 4 bytes
  - **Rango:** -2,147,483,648 a 2,147,483,647
  - **Formato:** enteros
  - **Ejemplo:**
    ```
    cantidad integer
    ```
  - **Uso:** el más común, balance entre rango y rendimiento.
- **bigint (int8)**
  - **Capacidad:** 8 bytes
  - **Rango:** $\pm 9{,}22 \times 10^{18}$
  - **Formato:** enteros grandes
  - **Ejemplo:**
    ```
    id bigint
    ```
  - **Uso:** valores muy grandes o identificadores masivos.

I-C. 2. Tipos autoincrementables

- **smallserial, serial, bigserial**
  - **Capacidad:** equivalente a smallint, integer y bigint

- **Formato:** enteros autogenerados
- **Ejemplo:**

  ```
  id serial primary key
  ```
- **Documentación:** estos tipos no son reales, sino una forma de crear una secuencia que genera valores automáticamente [2].

I-D. 3. Números exactos (decimal / numeric)

- **numeric(p,d) / decimal(p,d)**
  - **Formato de representación:** `numeric(precisión, escala)`
  - **Ejemplo:**

    ```
    precio numeric(10,2)
    ```
  - **Capacidad:**
    - Hasta 131072 dígitos antes del punto decimal
    - Hasta 16383 dígitos después del punto decimal
  - **Representación:** números con punto decimal (ej. 25.67)
  - **Comportamiento:**
    - Redondea si se exceden los decimales
    - Genera error si el número excede la precisión
  - **Ejemplo práctico:**

    ```
    insert into libros (precio)
    values (25.6666); -- se guarda como 25.67
    ```
  - **Uso:** datos financieros o donde se requiere exactitud.
  - **Documentación:** este tipo garantiza resultados exactos en operaciones aritméticas, aunque es más lento que otros tipos [2].

I-E. 4. Números aproximados (punto flotante)

- **real (float4)**
  - **Capacidad:** 4 bytes
  - **Rango:** $1E^{-37}$ a $1E^{+37}$
  - **Precisión:** $\sim$6 dígitos
  - **Formato:** notación científica (ej. 1.2E+3)
  - **Ejemplo:**

    ```
    temperatura real
    ```
- **double precision (float8)**
  - **Capacidad:** 8 bytes
  - **Rango:** $1E^{-307}$ a $1E^{+308}$
  - **Precisión:** $\sim$15 dígitos
  - **Formato:** notación científica

- **Ejemplo:**

  `medicion double precision`

Nota importante: Estos tipos no almacenan valores exactos, sino aproximaciones, por lo que pueden existir pequeñas diferencias en cálculos. También pueden manejar valores especiales como `Infinity`, `-Infinity` y `NaN` [2].

I-F. 5. Validaciones y comportamiento general

PostgreSQL aplica reglas automáticas al trabajar con datos numéricos:

- Si el valor está fuera de rango, se genera un error.
- Si se introduce una cadena numérica (ej. `'12.22'`), se convierte automáticamente.
- Si contiene caracteres inválidos, la operación falla.
- En tipos exactos, los valores se redondean si es necesario.

Por ejemplo, en un campo `decimal(5,2)`, el valor máximo permitido es 999.99, por lo que intentar insertar un valor mayor genera un error [1].

II. Tipos de datos de carácter en PostgreSQL

Para almacenar información textual en PostgreSQL se utilizan los tipos de datos de carácter, los cuales permiten guardar cadenas como nombres, direcciones, códigos o incluso números que no se usan para cálculos. Estas cadenas se representan mediante texto entre comillas simples [4], [3].

II-A. Tipos principales

PostgreSQL cuenta con tres tipos principales para almacenar texto:

II-A1. VARCHAR / CHARACTER VARYING:

- **Nombre:** `varchar(n)` o `character varying(n)`
- **Tipo:** longitud variable con límite
- **Capacidad:** de 1 hasta 10,485,760 caracteres
- **Formato de representación:** cadenas entre comillas simples

Ejemplo:

```
nombre varchar(30)
```

Uso: se utiliza cuando la longitud del texto puede variar (por ejemplo, nombres o direcciones).

Comportamiento:

- Si se excede la longitud definida, se genera un error.
- Solo almacena los caracteres necesarios (no rellena espacios).

II-A2. CHAR / CHARACTER:

- **Nombre:** `char(n)` o `character(n)`
- **Tipo:** longitud fija
- **Capacidad:** de 1 hasta 10,485,760 caracteres
- **Formato de representación:** cadenas de tamaño fijo

Ejemplo:

```
sexo char(1)
```

Uso: cuando todos los valores tienen la misma longitud (por ejemplo, códigos).
Comportamiento:

- Si la cadena es más corta, se rellena con espacios.
- Si es más larga, se genera un error.

II-A3. TEXT:

- **Nombre:** `text`
- **Tipo:** longitud variable ilimitada
- **Capacidad:** hasta aproximadamente 1 GB
- **Formato:** texto libre

Ejemplo:

```
descripcion text
```

Uso: para textos largos sin límite definido.
Nota: es el tipo más flexible y común en PostgreSQL [3].

II-B. Alias y equivalencias

PostgreSQL permite usar distintos nombres para los mismos tipos:

- `varchar(n)` ≡ `character varying(n)`
- `char(n)` ≡ `character(n)`

Esto facilita la compatibilidad con el estándar SQL [3].

II-C. Formato de representación

Las cadenas de caracteres siempre se escriben entre comillas simples:

```
'Juan Perez'
'Av. Reforma 123'
```

También es posible ingresar valores numéricos en campos de texto; PostgreSQL los convierte automáticamente a cadena:

```
telefono varchar(10)
insert into tabla values (12345); -- se guarda como '12345'
```

II-D. Validaciones y comportamiento

- Si una cadena supera la longitud definida, se genera un error.
- Si se convierte un número a texto y excede el tamaño, también falla.
- En `char(n)`, los espacios al final no afectan las comparaciones.
- En `varchar` y `text`, los espacios sí son significativos.

Ejemplo práctico:

```
sexo char(1)

insert into visitantes values ('masc');
-- ERROR: valor demasiado largo para char(1)
```

II-E. Capacidad y almacenamiento

- Cadenas cortas: 1 byte extra + longitud del texto
- Cadenas largas: 4 bytes de sobrecarga
- Tamaño máximo: aproximadamente 1 GB

PostgreSQL puede comprimir automáticamente cadenas largas para optimizar espacio [3].

II-F. Tipos especiales de carácter

Existen otros tipos menos comunes:

- **çhar"**: almacena un solo carácter (1 byte), usado internamente.
- **name**: hasta 64 bytes, utilizado para nombres de objetos en el sistema.

Estos tipos no se usan normalmente en aplicaciones [3].

III. Tipos de datos de fecha y hora en PostgreSQL

Los tipos de datos de fecha y hora en PostgreSQL permiten almacenar información temporal como fechas, horas o combinaciones de ambas. Estos tipos son muy importantes cuando se necesita registrar eventos, horarios o históricos dentro de una base de datos [6].

III-A. Tipos principales

PostgreSQL ofrece varios tipos para manejar fechas y tiempo:

III-A1. DATE:

- **Descripción:** almacena únicamente la fecha
- **Capacidad:** 4 bytes
- **Rango:** desde 4713 a.C. hasta 5,874,897 d.C.
- **Formato de representación:** `'AAAA-MM-DD'`

Ejemplo:

```
fecha date

insert into empleados values ('2008-12-31');
```

III-A2. TIME:

- **Descripción:** almacena solo la hora del día
- **Capacidad:** 8 bytes
- **Rango:** 00:00:00 a 24:00:00
- **Precisión:** hasta microsegundos
- **Formato:** `'HH:MM:SS'`

Ejemplo:

```
hora time

insert into asistencia values ('13:00:59');
```

III-A3. TIMESTAMP:

- **Descripción:** almacena fecha y hora
- **Capacidad:** 8 bytes
- **Rango:** igual que DATE
- **Precisión:** hasta microsegundos
- **Formato:** `'AAAA-MM-DD HH:MM:SS'`

Ejemplo:

```
fechahora timestamp

insert into asistencia values ('2008-12-31 13:00:00');
```

III-A4. INTERVAL:

- **Descripción:** almacena periodos de tiempo
- **Capacidad:** 12 bytes
- **Rango:** hasta millones de años
- **Formato:** combinación de unidades (años, días, horas)

Ejemplo:

```
duracion interval

select interval '2 days 3 hours';
```

III-B. Formato de representación

Las fechas y horas siempre se escriben entre comillas simples. PostgreSQL acepta distintos formatos, pero el más recomendado es el estándar ISO 8601:

```
'2008-12-31'
'2008-12-31 13:00:00'
```

También se pueden usar otros separadores como '/', '-' o '.', dependiendo de la configuración [4].

III-C. Configuración del formato (DateStyle)

PostgreSQL permite cambiar el formato de entrada de fechas mediante la variable `DateStyle`.

Ejemplo:

```
SET datestyle TO 'European';

insert into asistencia
values ('21/12/2018 13:00:00');
```

Opciones comunes:

- ISO (formato estándar)
- European (dd/mm/aaaa)
- US (mm/dd/aaaa)
- German (dd.mm.aaaa)

Esto permite adaptar el formato según la región [6].

III-D. Zonas horarias

PostgreSQL maneja dos tipos de timestamp:

- **timestamp sin zona horaria**
- **timestamp con zona horaria (timestamptz)**

Comportamiento:

- Los valores con zona horaria se almacenan internamente en UTC.
- Al mostrarse, se convierten a la zona horaria del sistema.

Ejemplo:

```
timestamp with time zone '2004-10-19 10:23:54+02'
```

III-E. Valores especiales

PostgreSQL permite usar valores especiales de fecha y hora:

- `'now'` → fecha y hora actual
- `'today'` → fecha actual
- `'tomorrow'` → día siguiente
- `'yesterday'` → día anterior
- `'infinity'` y `'-infinity'`

Estos valores facilitan operaciones sin necesidad de escribir fechas exactas [6].

III-F. Validaciones y comportamiento

- Las fechas deben ir entre comillas simples.
- PostgreSQL acepta múltiples formatos de entrada.
- Si el formato es ambiguo, se interpreta según `DateStyle`.

- Los valores fuera de rango generan error.

Ejemplo práctico:

```
insert into asistencia
values ('11111111','2008/12/31 13:00:00');

set datestyle to 'European';

insert into asistencia
values ('22222222','21/12/2018 13:00:00');
```

IV. Tipos de datos monetarios en PostgreSQL

PostgreSQL cuenta con un tipo de dato específico para manejar valores de dinero, el cual permite almacenar cantidades monetarias de forma directa. Este tipo es útil cuando se requiere representar valores financieros, aunque debe usarse con cuidado dependiendo del caso [7].

IV-A. Tipo principal

IV-A1. MONEY:

- **Nombre:** `money`
- **Capacidad:** 8 bytes
- **Rango:**

$$-92233720368547758{,}08 \;\; a \;\; +92233720368547758{,}07$$

- **Precisión:** fraccionaria fija (generalmente 2 decimales)

IV-B. Formato de representación

Los valores monetarios se pueden ingresar en distintos formatos:

- Números enteros: `1000`
- Números decimales: `1000.50`
- Formato monetario: `'$1,000.50'`

Ejemplo:

```
precio money

insert into productos values ('$1,250.75');
```

Nota: El formato de salida depende de la configuración regional (`lc_monetary`), por lo que puede variar según el país (símbolo de moneda, separadores, etc.) [7].

IV-C. Comportamiento y configuración

- La precisión (cantidad de decimales) depende de la configuración regional.
- El formato de salida puede cambiar entre sistemas si la configuración es diferente.
- Para evitar problemas al migrar bases de datos, se recomienda mantener la misma configuración `lc_monetary`.

IV-D. Conversiones entre tipos

El tipo `money` puede convertirse a otros tipos y viceversa:

- Desde `integer`, `bigint` o `numeric` → sin problema.
- Desde `float` o `double` → no recomendado (puede haber errores).

Ejemplo:

```
select '12.34'::float8::numeric::money;
```

También se puede convertir de `money` a `numeric` sin pérdida de precisión:

```
select '52093.89'::money::numeric;
```

IV-E. Operaciones con valores monetarios

- Al dividir un valor `money` entre un entero, se trunca la parte decimal.
- Para obtener resultados más precisos, se recomienda convertir a `numeric`.

Ejemplo:

```
-- Puede perder precisión
select precio / 2 from productos;

-- Forma recomendada
select (precio::numeric / 2)::money from productos;
```

Nota importante:

- Dividir `money` entre `money` devuelve un valor numérico (double precision).
- Las unidades monetarias se cancelan en la operación.

IV-F. Buenas prácticas

- No usar tipos de punto flotante para manejar dinero.
- Preferir `numeric` cuando se necesite mayor control y precisión.
- Usar `money` cuando se requiera simplicidad y formato automático.

V. Tipos de datos booleanos en PostgreSQL

El tipo de dato booleano en PostgreSQL se utiliza para representar valores lógicos, es decir, estados de verdadero o falso. Este tipo es muy útil en situaciones donde se requiere almacenar condiciones, como estados activos/inactivos, respuestas de sí/no o validaciones [8].

V-A. Tipo principal

V-A1. BOOLEAN:

- **Nombre:** `boolean`
- **Capacidad:** 1 byte
- **Descripción:** almacena valores lógicos

V-B. Valores posibles

El tipo booleano puede tener tres estados:

- **TRUE**: verdadero
- **FALSE**: falso
- **NULL**: valor desconocido o no definido

Esto es importante porque no solo se manejan dos estados, sino también un tercero que representa la ausencia de información.

V-C. Formato de representación

PostgreSQL permite ingresar valores booleanos de distintas formas:

Para TRUE:

- `true`
- `yes`
- `on`
- `1`

Para FALSE:

- `false`
- `no`
- `off`
- `0`

También se aceptan abreviaciones como `t` o `f`, y no importa si se usan mayúsculas o minúsculas. Los espacios antes o después del valor son ignorados [8].

Salida: Internamente PostgreSQL muestra los valores como:

- `t` para verdadero
- `f` para falso

V-D. Ejemplos de uso

```
create table prueba (
    activo boolean,
    descripcion text
);
```

```
insert into prueba values (true, 'activo');
insert into prueba values (false, 'inactivo');

select * from prueba;
```

Resultado:

```
 t | activo
 f | inactivo
```

También se puede filtrar directamente:

```
select * from prueba where activo;
```

V-E. Conversiones y comportamiento

- PostgreSQL reconoce automáticamente `TRUE` y `FALSE` como booleanos.
- El valor `NULL` no tiene tipo específico, por lo que en algunos casos se debe convertir explícitamente:

```
select NULL::boolean;
```

También se pueden convertir cadenas a booleano:

```
select 'yes'::boolean;
```

Nota: El sistema puede deducir automáticamente el tipo en muchos casos, por lo que no siempre es necesario hacer conversiones explícitas [8].

VI. Tipos de datos de cadenas de bits en PostgreSQL

Los tipos de cadenas de bits en PostgreSQL permiten almacenar secuencias de valores binarios formadas únicamente por 0 y 1. Estos tipos son útiles cuando se necesita trabajar con máscaras de bits, configuraciones binarias o información a nivel de bits [9].

VI-A. Tipos principales

PostgreSQL define dos tipos de cadenas de bits:

VI-A1. BIT(n):

- **Nombre:** `bit(n)`
- **Tipo:** longitud fija
- **Capacidad:** exactamente n bits
- **Formato de representación:** cadenas binarias con prefijo `B`

Ejemplo:

```
codigo bit(3)

insert into tabla values (B'101');
```

Comportamiento:

- La cadena debe tener exactamente la longitud definida.
- Si tiene menos o más bits, se genera un error.

VI-A2. BIT VARYING(n):

- **Nombre:** `bit varying(n)`
- **Tipo:** longitud variable
- **Capacidad:** hasta n bits
- **Formato:** secuencia de bits

Ejemplo:

```
flags bit varying(5)

insert into tabla values (B'101');
```

Comportamiento:

- Permite cadenas de distinta longitud.
- Si se supera el límite, se genera un error.

VI-B. Valores por defecto

- `bit` sin especificar longitud ⇒ equivale a `bit(1)`
- `bit varying` sin longitud ⇒ longitud ilimitada

VI-C. Formato de representación

Las cadenas de bits se representan con el prefijo `B` seguido de la secuencia binaria entre comillas:

```
B'101'
B'00011'
```

VI-D. Conversiones y comportamiento

PostgreSQL permite convertir valores de cadenas de bits de forma explícita:

- Si se convierte a `bit(n)`:
  - Se rellena con ceros a la derecha si es más corta
  - Se trunca si es más larga
- Si se convierte a `bit varying(n)`:
  - Se trunca si excede la longitud máxima

Ejemplo:

```
insert into test values (B'10'::bit(3));
-- resultado: 100
```

VI-E. Ejemplo completo

```
create table test (
    a bit(3),
    b bit varying(5)
);

insert into test values (B'101', B'00');

-- Error: longitud incorrecta
insert into test values (B'10', B'101');

-- Corrección con conversión
insert into test values (B'10'::bit(3), B'101');

select * from test;
```

VI-F. Capacidad y almacenamiento

- Se utiliza 1 byte por cada 8 bits almacenados
- Se agregan entre 5 y 8 bytes adicionales de sobrecarga
- Las cadenas largas pueden comprimirse automáticamente

VI-G. Uso y documentación

- Permiten trabajar directamente con datos binarios a nivel de bits
- Son útiles para máscaras, flags o configuraciones binarias
- PostgreSQL incluye operadores lógicos de bits para manipular estos datos

VI-H. Conclusión

Los tipos de cadenas de bits en PostgreSQL permiten almacenar información binaria de forma eficiente, ya sea con longitud fija o variable. Aunque no son tan comunes como otros tipos, resultan muy útiles en aplicaciones donde se manejan datos a nivel de bits, como sistemas de control o configuraciones internas.

REFERENCIAS

[1] TutorialesProgramacionYa.com, "Tipos numéricos en PostgreSQL," 2026. [En línea]. Disponible en: https://www.tutorialesprogramacionya.com/postgresqlya/temarios/descripcion.php?cod=174punto=16inicio=. [Accedido: 14-abr-2026].

[2] PostgreSQL Global Development Group, "PostgreSQL 17.2 Documentation: 8.1. Numeric Types," Feb. 26, 2026. [En línea]. Disponible en: https://www.postgresql.org/docs/current/datatype-numeric.html. [Accedido: 14-abr-2026].

[3] PostgreSQL Global Development Group, "PostgreSQL 17.2 Documentation: 8.3. Character Types," Feb. 26, 2026. [En línea]. Disponible en: https://www.postgresql.org/docs/current/datatype-character.html. [Accedido: 14-abr-2026].

[4] TutorialesProgramacionYa.com, "Tipos de texto en PostgreSQL," 2026. [En línea]. Disponible en: https://www.tutorialesprogramacionya.com/postgresqlya/temarios/descripcion.php?cod=173punto=15inicio=. [Accedido: 14-abr-2026].

[5] TutorialesProgramacionYa.com, "Tipos de fecha y hora en PostgreSQL," 2026. [En línea]. Disponible en: https://www.tutorialesprogramacionya.com/postgresqlya/temarios/descripcion.php?cod=175punto=17inicio=. [Accedido: 14-abr-2026].

[6] PostgreSQL Global Development Group, “PostgreSQL 8.2 Documentation: 8.5. Date/Time Types,” Dec. 5, 2011. [En línea]. Disponible en: https://www.postgresql.org/docs/8.2/datatype-datetime.html. [Accedido: 14-abr-2026].

[7] PostgreSQL Global Development Group, “PostgreSQL 17.2 Documentation: 8.2. Monetary Types,” Feb. 26, 2026. [En línea]. Disponible en: https://www.postgresql.org/docs/current/datatype-money.html. [Accedido: 14-abr-2026].

[8] PostgreSQL Global Development Group, “PostgreSQL 17.2 Documentation: 8.6. Boolean Type,” Feb. 26, 2026. [En línea]. Disponible en: https://www.postgresql.org/docs/current/datatype-boolean.html. [Accedido: 14-abr-2026].

[9] PostgreSQL Global Development Group, “PostgreSQL 17.2 Documentation: 8.10. Bit String Types,” Feb. 26, 2026. [En línea]. Disponible en: https://www.postgresql.org/docs/current/datatype-bit.html. [Accedido: 14-abr-2026].